Advanced Combustion via Microgravity Experiments (ACME)
Structure and Response of Spherical Diffusion Flames (s-Flame)

ACME/s-Flame Experimental Procedure

Prepared by:
Dennis P. Stocker*, ACME Project Scientist*
NASA Glenn Research Center
dennis.p.stocker@nasa.gov, 216-433-2166



Table of Contents

	Page
1. Introduction	**2**
2. Test Sequence and Parameters	**5**
3. s-Flame Script	**12**
4. Test Matrices	**36**
A. Test Matrices: Years in Advance	36
B. Test Matrices: Months in Advance	37
C. Test Matrices: Days in Advance	37
D. Test Matrices: As-Run	38
5. Nomenclature	**38**

Identification of commercial products is for clarification and does not indicate endorsement by NASA or the federal government.

1. Introduction

The narrative description of the experimental procedure is an excerpt from the 'Methods / Experimental' section of the following conference paper.

Y.-C. Chien, J. Tinajero, D. Stocker, U. Hegde, and D. Dunn-Rankin, "Electric Field Effects on Flames in Microgravity on the International Space Station," 2018 Spring Technical Meeting of the Central States Section of The Combustion Institute, Minneapolis, MN, 20-22 May 2018.

While the paper was written for the Electric-Field Effects on Laminar Diffusion Flames (E-FIELD Flames) experiment, much of the discussion applies to ACME in general, including the s-Flame investigation. The figure referenced in this excerpt is not included herein. Clarifications and additional information, not in the conference paper, are included in boxed notes like this.

The CIR experiments are configured, but not conducted, by the ISS crew. Thus far, ACME's activity has included the majority of the crew members, regardless of their nationalities. Most often the astronauts and cosmonauts exchange gas bottles, mass flow controllers, and igniter tips. In the latter case, the igniter tip exchange can occur after failure or proactively before failure when the crew already has changes to make changes in the interior of the insert. While these activities may seem mundane, the crew play a critical role in setting up the hardware for each experiment. As an extreme example, there were crew activities on 8 separate days in Sept.-Oct. 2017 to configure the CIR and ACME hardware for the first ACME experiment, namely the Coflow Laminar Diffusion Flame (CLD Flame) experiment. In contrast, only two crew days were required to reconfigure the hardware for the first E-FIELD Flames tests because of the initial use of the same coflow burner. Planning is critical because crew activity is scheduled three weeks in advance, for which detailed procedures must be prepared and uplinked to the ISS.

Similarly, the s-Flame experiment was conducted using the same burner as the Flame Design investigation which preceded it. However, there was hardware reconfiguration for s-Flame to enable use of an additional intensified camera for imaging OH chemiluminescence, namely the Low Light Level Ultraviolet (LLL-UV) camera.*

All CIR tests are remotely commanded from the ground from NASA's Glenn ISS Payload Operations Center (GIPOC) in Cleveland, Ohio. Typically, four to six people staff four consoles during the testing. The CIR hardware is controlled from the Rack Officer (RO) and Data Management Officer (DMO) consoles, and the ACME hardware is controlled from the Payload Developer (PD) console. Two shifts of personnel are scheduled to staff these three consoles on test days because they typically last 14 hours with the many pre- and post-testing activities. Given that the tests are remotely commanded, the operations are carried out in the day and evening in Cleveland, e.g., from 7 AM to 9 PM. A day's tests, i.e., with flames, are normally conducted in a period of 10 hours of less. Tests are normally conducted on three weekdays per two weeks. On intervening weekdays, the image data is downlinked and the crew often swap one or both gas bottles. Transferring and downlinking data from a day's tests also requires a 2-shift day for the RO and DMO consoles, where it is always necessary to verify that data was downlinked without errors and that data is not missing. Those two positions are also staffed to guide and oversee the crew activity. As a result, the CIR consoles are normally actively staffed on four weekdays per week. A team of four carry out the detailed planning for the CIR and ACME console staff, and the daily operational plans typically exceed 50 pages on days including testing.

The science console allows for review of the analog video and non-imaging telemetry (at 1 Hz) in near real time but doesn't allow for commanding. In the case of E-FIELD Flames, arrangements were made so that

the analog video is streamed to a secure computer at UC Irvine to allow the research team to participate in test operations and to recommend adjustments to the day's planned parameters. That team does not see the telemetry during the testing, but is in communication via phone and e-mail with their NASA research partners at the GRC ISS Payload Operations Center (GIPOC). While there are many preparatory steps, the tests are ultimately conducted using an uploaded software script and parameter files. In advance of the test day, the UC Irvine science team plans the nearly 60 parameters in the voltage sweep script and specifies other parameters that are configured outside of the script, such as camera settings. It is also possible to effectively conduct tests in a 'manual' mode, where parameters are changed in response to the analog video downlink or telemetry. For example, the electric field potential can be adjusted to approach a stability or soot limit.

Provisions have been made to enable the investigators from all ACME experiments to remotely view the analog video while they communicated verbally with NASA science personnel in the operations center. Thus far, all ACME flame tests, including those for s-Flame and E-FIELD Flames, have been conducted in an automated manner using software scripts and extensive parameter files.

Typically, ten E-FIELD Flames tests are conducted per test day. Despite the short test durations, the low rate results from several unavoidable factors. After a test is conducted, the 125-Hz non-imaging data is downlinked for immediate review by the science team. A small sample of images are often downlinked to ensure that the camera exposures are appropriate, where the times for those images are typically selected based on non-imaging data to provide extreme examples. The review helps to verify parameters or plan for adjustments in subsequent tests, e.g., to select the voltage range after determining where the ion current saturates for the given flame conditions. After a test is conducted, a fan circulates the chamber atmosphere after which the chamber is vented (to space) back to atmospheric pressure, if needed. Midway through a day's E-FIELD Flames tests, the atmosphere is circulated for 15 minutes through a filter cartridge filled with molecular sieves to capture any soot and adsorb at least a portion of the combustion products, e.g., carbon dioxide, water vapor, and incomplete combustion products. The tests are only conducted when both Ku and S band communication is available through NASA's Tracking and Data Relay Satellite (TDRS) System. Depending on the various orbits, there are periodic communication outages with a loss of signal (LOS) that can be a short handover (from one satellite to another) or can last as long as 30 minutes.

In contrast to E-FIELD Flames, an average of 15 flames were ignited per test day for s-Flame, but they were of even shorter durations. The pressure rise with an s-Flame test was small and the chamber was rarely vented after a test unless the chamber atmosphere was being adjusted for other purposes. In this regard, the chamber atmosphere was sometimes scrubbed for the s-Flame investigation by recirculating it through the adsorber cartridge, but this was not a regular routine as in the case of E-FIELD Flames. The chamber was also sometimes partially or completely vented and refilled during s-Flame operations, for example to adjust the atmosphere's oxygen concentration.

Unlike E-FIELD Flames, the s-Flame experiment also made use of the CIR's Gas Chromatograph Instrumentational Package (GCIP) to measure species concentrations of the atmosphere in the combustion chamber. There is no sampling probe and measurements are only made before, between, or after tests when the chamber atmosphere was 'stirred' through use of a fan. For s-Flame, the GC measurements were made on most but not all test days, but they were limited in scope because of the significant time required. The GC measurements were sometimes only made at the end of the test day but were occasionally made earlier in the day, for example prior to an adjustment of the chamber atmosphere.

A variety of constraints limit the number of tests that can be performed, both overall and on a daily basis. The next combustion experiments in the CIR will feature solid fuels and focus on flame spread and material flammability. That follow-on plan limits the duration of ACME operations, where each ACME experiment is allotted four months for testing. Although it requires additional crew time, these allotments are split into two rounds so that the investigators have a break from testing for careful review of the early results so that they can maximize the scientific outcome of their second round. This paper reports on early results from the first round of E-FIELD Flames.

In contrast to earlier plans, the two rounds of the s-Flame experiment occurred sequentially because of limited crew availability. Due to delays in commercial crew transport, the space station had three rather six crew members through much of 2020. As a result of the reduced crew size, the s-Flame investigators did get a 3-month break (in Feb. to April) between their two rounds of flame tests allowing them to study early results and prepare for the second round. Hardware issues also stretched out s-Flame operations in the latter half of 2019.

While nitrogen is available from the space station, the combustion testing is limited by the available gas bottles that were and are launched to support the tests. For E-FIELD Flames and most of ACME's experiments, the test matrices used to plan those gas resources were established in 2010. At that time, NASA limited the scope of the ACME tests matrices with the intent of focused experiments with limited objectives rather than comprehensive experiments. As such, the general target was 50 tests per experiment. For most ACME experiments, the limiting gas constraint is the oxygen/nitrogen bottles used to fill the chamber. This resource constraint is exacerbated by the CIR because it is equipped with a pair of redundant safety timers to limit the fuel that can flow into the combustion chamber (where the timers can be set for the oxidizer flow in the case of inverse flame testing). The timers can only be reset by venting the combustion chamber. Therefore, the timer limit combined with other constraints effectively corresponds to the day's maximum flow time for the fuel (or more generally, the limiting reactant). For E-FIELD Flames, the safety timers are set to the maximum value of 999 seconds, which corresponds to the 10 tests typically completed per day.

The second round of the s-Flame experiment featured helium dilution of both the fuel and the oxidizer, so its testing was also limited by the oxygen/helium bottles launched for that purpose.

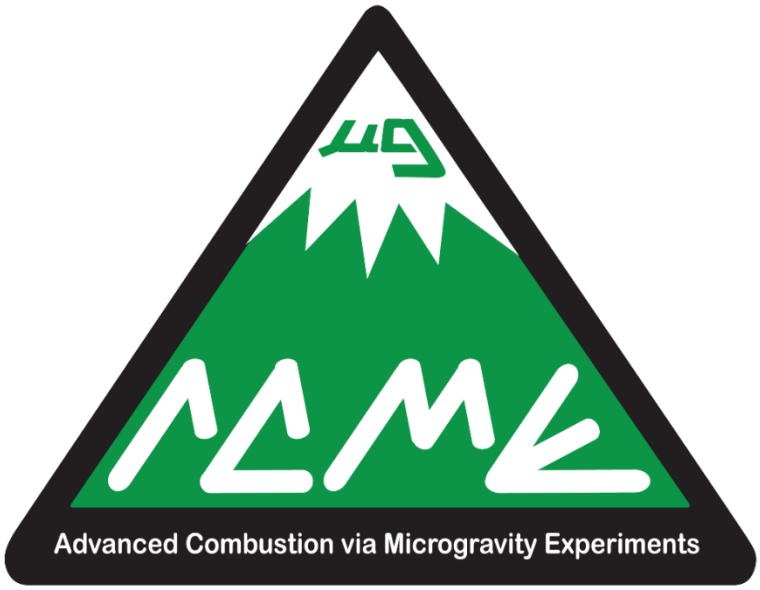
The 999-second limit on the valve safety timers has thus far been common for all ACME testing. However, the timers can be reset to 999 by nearly completely venting the chamber. Sufficient time and gas resources are needed to vent and refill the chamber and still conduct tests afterward and that was only occasionally done for s-Flame.

Another constraint is the available time in a day, where the console personnel and ISS resources, such as the video downlink, are planned for a limited number of hours. When necessary for the research, NASA does have continuous 24-hour/day operations for some non-CIR experiments. But as discussed, the ACME tests are typically two minutes or less in duration, so the testing is almost always limited to the planned durations.

Data limits are yet another constraint, where the on-orbit hardware can store a limited amount of data. More importantly, 127,000 MB is the most that can be reasonably managed in a two-shift transfer and downlink day. Therefore, tests days are limited to this amount of data, where image data from select

cameras is often archived at reduced rates to mitigate this challenge. Nonetheless, the limit sometimes forces decisions to abandon recorded images.

This detailed description of operations and hardware provides the context under which the ACME experiments are conducted. …



2. Test Sequence and Parameters

All s-Flames tests conducted on the ISS were carried out using automated software which had its roots in ACME's Integrated Science Requirements Documents (ISRD, RDR version, Rev. B) and Science Operations Requirements Documents (SORD, Rev. A). The s-Flame Science Requirements Document (SRD) is of course pertinent but only that which was incorporated in the ISRD and SORD was used by the engineering team in developing the control software.

The major phases of the test sequence are graphically displayed in Figures 1 and 2, which are respectively for the two-rounds of s-Flame testing. The first round was carried out with nitrogen dilution of the fuel and atmosphere, where that inert gas was available from the space station. Meanwhile, the second round was carried out with fuel diluted in advance (on the ground) with helium. Similarly, bottles of 30% oxygen with a balance of helium (on a volume basis) were launched for this testing, where that was often diluted with 100% helium (also launched for s-Flame) to conduct tests in a 21% oxygen atmosphere.

The lines appearing in the Figures 1 and 2 represent the inert (i.e., nitrogen or helium) flow rate, the fuel flow rate, the igniter position and power. The lines that are more lightly shaded represent some optional changes to flow rates, for example for a change in flow rate(s) following ignition. However, the flow variations in the darkly shaded lines are also optional. The phases are described in detail at the bottom of each schematic, where the nomenclature throughout these figures generally corresponds to the script parameter names.

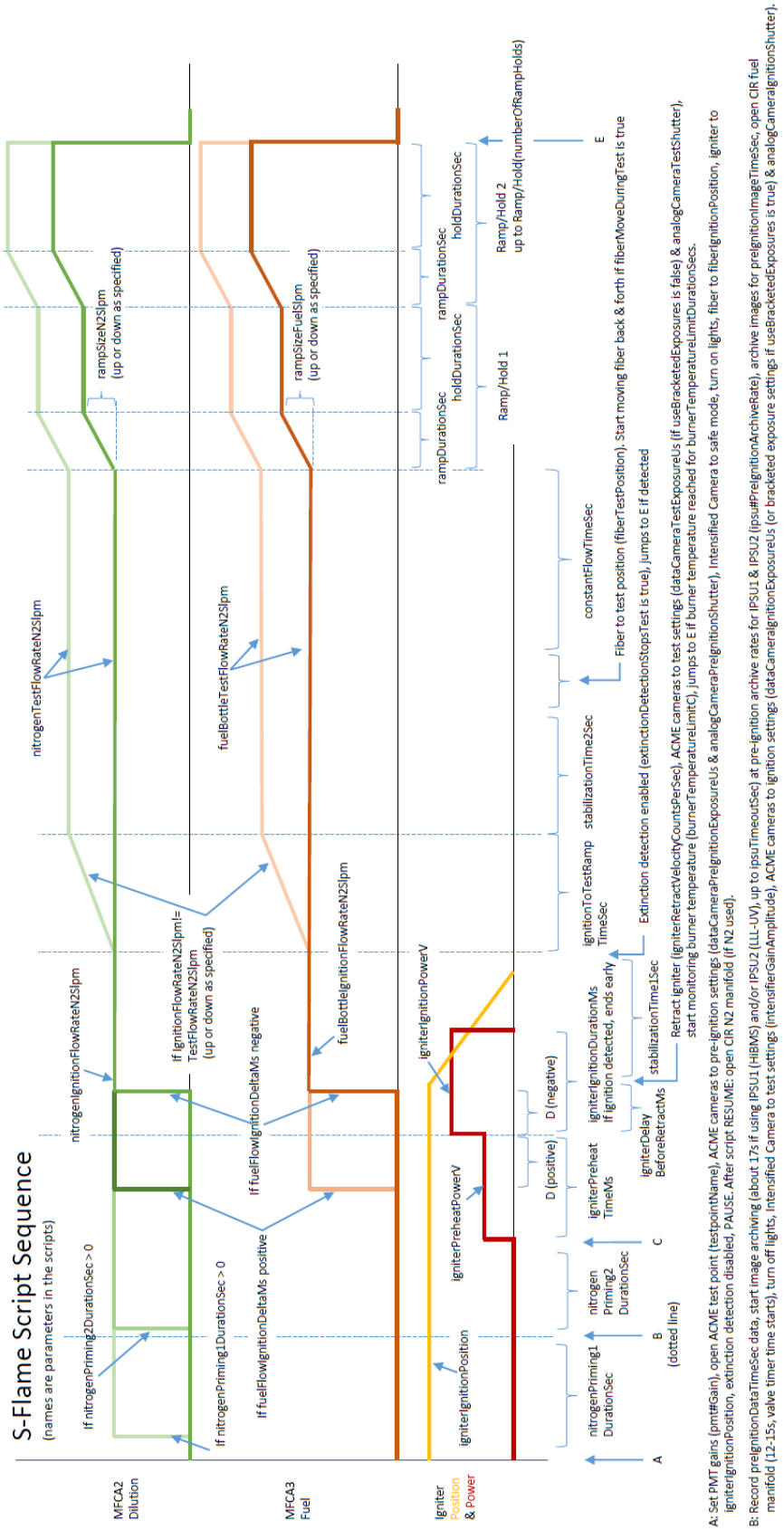


Figure 1. Schematic illustrating the s-Flame script sequence in round 1 testing (featuring dilution with nitrogen).

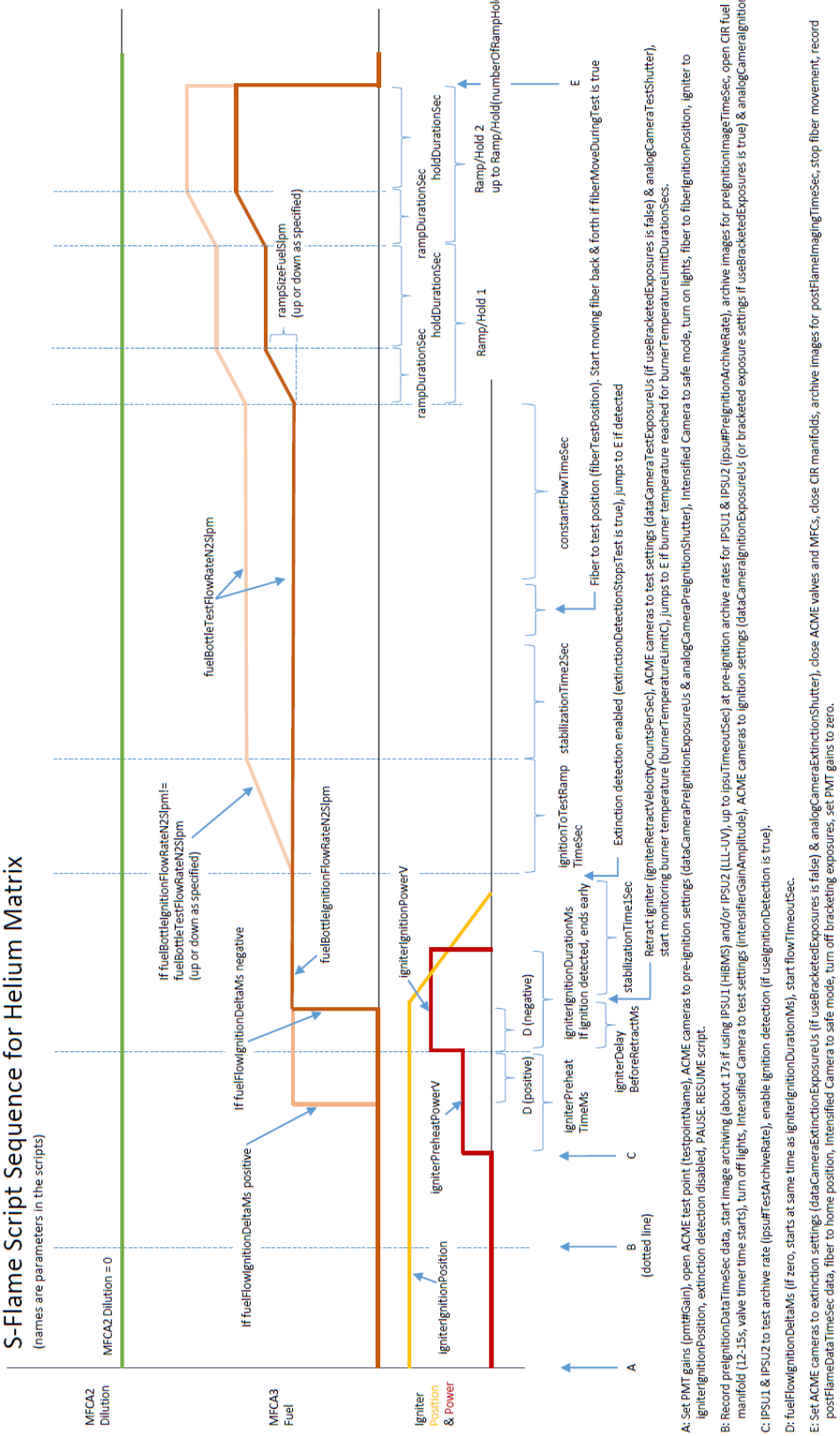


Figure 2. Schematic illustrating the s-Flame script sequence during the second round of testing, featuring helium dilution. For this testing, the fuel was diluted on the ground and launched in premixed bottles. As a result, the helium flow is zero.

The s-Flame script uses 68 parameters which are listed below with example values from a round-2 test, 20190A1. The parameters are briefly explained, where many of them are common to all ACME experiments because of their use of the same set of modular hardware. The lines separate general types of parameters, e.g., those corresponding to imaging, ignition, the fiber array, gas flow, etc. Parameters highlighted in yellow were set to zero (i.e., not used) in round 2 testing (as in this example) when there was no nitrogen flow. There are no corresponding helium parameters because the fuel and helium was premixed on the ground.

---

testpointName:20190A1,
test point name that will be used by the IPSUs and ACME Data File (in the image and data file names and directories)

---

preIgnitionDataTimeSec:0,
time (in seconds) ACME Data File records data before IPSUs open test point and start archiving images
postFlameDataTimeSec:15,
time (in seconds) ACME Data File records data after the postFlameImagingTimeSec has elapsed which occurs after command sent to close manifolds (i.e., stop gas flow)

---

useIPSUs:true,
indicates if IPSUs will be used (images archived) for the test point
ipsuTimeoutSec:40,
time (in seconds) to wait for command responses from IPSUs and IPSUG when opening and starting test points
useAcmeCamera:true,
indicates if the ACME Camera will be used for the test point
useIntensifiedCamera:true,
indicates if the CIR Intensified Camera will be used for the test point
ipsu1PreIgnitionArchiveRate:100,
indicates that every nth HiBMS Camera (IPSU1) image is saved (archived) (e.g., 10 indicates every 10th image) through nitrogenPriming2DurationSec; if nitrogenPriming2DurationSec is zero, rate is used until igniter powered or fuel flow starts
ipsu2PreIgnitionArchiveRate:100,
indicates that every nth LLL-UV Camera (IPSU2) image is saved (archived) (e.g., 10 indicates every 10th image) through nitrogenPriming2DurationSec; if nitrogenPriming2DurationSec is zero, rate is used until igniter powered or fuel flow starts
ipsu1TestArchiveRate:2,
indicates that every nth HiBMS Camera (IPSU1) image is saved (archived) (e.g., 10 indicates every 10th image) after nitrogenPriming2DurationSec is complete; if nitrogenPriming2DurationSec is zero, rate is used before igniter powered or fuel flow starts
ipsu2TestArchiveRate:1,
indicates that every nth LLL-UV Camera (IPSU2) image is saved (archived) (e.g., 10 indicates every 10th image) after nitrogenPriming2DurationSec is complete; if nitrogenPriming2DurationSec is zero, rate is used before igniter powered or fuel flow starts
dataCameraPreIgnitionExposureUs:100000,
exposure (in microseconds) of the ACME Camera prior to ignition, only while the lights are on (set for imaging the igniter position)

dataCameraIgnitionExposureUs:100000,
exposure (in microseconds) of the ACME Camera during ignition and igniter retraction (set for imaging the ignition event), up to but not including stabilizationTime1Sec
dataCameraTestExposureUs:650000,
exposure (in microseconds) of the ACME Camera after igniter retraction (set for imaging the flame)
dataCameraExtinctionExposureUs:650000,
exposure (in microseconds) of the ACME Camera after extinction is detected, or numberOfRampHolds (if used) or constantFlowTimeSec is complete (set for post-flame imaging)
analogCameraPreIgnitionShutter:2,
shutter speed of the ACME Chamber Insert Camera (ops camera - no IPSU data recorded for this camera) prior to ignition, only while lights are on (set for imaging the igniter position)
analogCameraIgnitionShutter:2,
shutter speed of the ACME Chamber Insert Camera (ops camera - no IPSU data recorded for this camera) during ignition and igniter retraction (set for imaging the ignition event), up to but not including stabilizationTime1Sec
analogCameraTestShutter:2,
shutter speed of the ACME Chamber Insert Camera (ops camera - no IPSU data recorded for this camera) after igniter retraction (set for imaging the flame)
analogCameraExtinctionShutter:0,
shutter speed of the ACME Chamber Insert Camera (ops camera - no IPSU data recorded for this camera) after extinction is detected, or numberOfRampHolds (if used) or constantFlowTimeSec is complete (set for post-flame imaging)
intensifierGainAmplitude:15000,
CIR Intensified Camera intensifier gain setting; commanded to this value after turning off chamber LED lights; this is the actual gain value; the script will convert to counts when sending the command
preIgnitionImageTimeSec:0,
time (in seconds) IPSUs record images before the CIR fuel manifold is commanded open
postFlameImagingTimeSec:3,
time (in seconds) IPSUs record images after flow stops

---

useBracketedExposures:true,
indicates if bracketed exposures on the ACME Camera will be used for the test point; if true, starts before nitrogenPriming2DurationSec and replaces dataCameraIgnitionExposureUs, dataCameraTestExposureUs, dataCameraExtinctionExposureUs
bracketingImagesPerExposure:1,
number of images at each exposure before moving on to the next exposure when using bracketed exposure
bracketingNumberOfExposures:3,
number of exposures when using bracketed exposure
bracketingExposureStartUsec:60000,
initial exposure (in microseconds) for the ACME Camera when using bracketed exposure
bracketingExposureMultiplierTimes100:500,
value, divided by 100, that will get multiplied to the current exposure to determine the next exposure, when using bracketed exposure

---

igniterIgnitionPosition:2250,
position (in counts) of the igniter for preheat and ignition

igniterPreheatPowerV:3.5,
igniter power (in volts) during preheat (igniterPreheatDurationMs)
igniterPreheatTimeMs:2000,
time (in milliseconds) the igniter is powered during the preheat
igniterIgnitionPowerV:4.49,
igniter power (in volts) during ignition (igniterIgnitionDurationMs)
igniterIgnitionDurationMs:500,
time (in milliseconds) the igniter is powered for ignition; ends early if ignition is detected
igniterDelayBeforeRetractMs:500,
time (in milliseconds) the igniter stays in igniterIgnitionPosition before retracting
igniterRetractVelocityCountsPerSec:500,
velocity (in counts/second) used when the igniter retracts if ignition is not detected

---

useIgnitionDetection:true,
indicates if Ignition Detection will be used; when ignition is detected (per the specified values), the igniter turns off and retracts
extinctionDetectionStopsTest:true,
indicates if Extinction Detection will be used to detect flame extinction; when extinction is detected (per the specified values or when commanded by operators), the gas flows stop and script skips to the end; script still performs post-flame camera configurations, data and imaging collection, and then closes the test point nicely

---

pmt1Gain:5.5,
PMT1 gain during the script; set to 0 at end of script; PMT1 = 310nm, OH*
pmt2Gain:5,
PMT2 gain during the script; set to 0 at end of script; PMT2 = Broadband
pmt3Gain:5.5,
PMT3 gain during the script; set to 0 at end of script; PMT3 = 430nm, CH*

---

fiberIgnitionPosition:10000,
position (in counts) the fiber moves to prior to ignition (pre-positions the fiber so it can reach fiberTestPosition more quickly)
fiberTestPosition:14100,
position (in counts) the fiber moves to during test point (after stabalizationTime2Sec), starting position for fiber movement
fiberMoveDuringTest:false,
indicates if fiber will move back and forth during test; if true, fiber will move back and forth during the test; starts after fiberTestPosition is reached; if false, fiber will still go into test position (fiberTestPosition) but not move back and forth
fiberSegmentDistance:400,
distance (in counts) the fiber moves towards the burner from the fiberTestPosition and back
fiberVelocity:500,
velocity (in counts/second) of fiber when moving during test
fiberAcceleration:1000,
acceleration (in counts/second squared) of fiber when moving during test
fiberDeceleration:0,
deceleration (in counts/second squared) of fiber when moving during test
fiberHoldDurationSec:1,
time (in seconds) to hold at the innermost and outermost positions of the fiber movement

fiberCycles:100,
number of cycles the fiber moves; one cycle is towards the burner (number of counts in fiberSegmentDistance) and back (to fiberTestPosition)

---

useFoma:true,
indicates if gas flow will be used for test points
flowTimeoutSec:120,
total time (in seconds) gas flows during script, starting when the fuel flow starts; gas flow stops after this time if not at end of script
nitrogenPriming1DurationSec:0,
time (in seconds) for nitrogen to flow prior to the start of fuel flow; time is before IPSUs start archiving, before opening CIR fuel manifold, and before nitrogenPriming2DurationSec
nitrogenPriming2DurationSec:0,
time (in seconds) for nitrogen to flow prior to the start of fuel flow; time starts after CIR fuel manifold has been opened
fuelFlowIgnitionDeltaMs:1750,
time (in milliseconds) fuel (and nitrogen) flows before or after start of igniterIgnitionDurationMs
stabilizationTime1Sec:0,
time (in seconds) to wait after igniter starts retracting, before increasing fuel (and nitrogen) to test flow rates
stabilizationTime2Sec:0,
time (in seconds) to wait after ignitionToTestRampTimeSec, before the fiber moves into test position (fiberTestPosition)
fuelBottleIgnitionFlowRateN2Slpm:0.35671,
initial fuel bottle flow rate (SLPM N2 basis) through stabilizationTime1Sec
nitrogenIgnitionFlowRateN2Slpm:0,
initial nitrogen flow rate (SLPM N2 basis) through stabilizationTime1Sec
fuelBottleTestFlowRateN2Slpm:0.35671,
fuel bottle flow rate (SLPM N2 basis) after ignitionToTestRampTimeSec through the end of constantFlowTimeSec
nitrogenTestFlowRateN2Slpm:0,
nitrogen flow rate (SLPM N2 basis) after ignitionToTestRampTimeSec through the end of constantFlowTimeSec
ignitionToTestRampTimeSec:0,
time (in seconds) it takes to ramp from fuelBottleIgnitionFlowRateN2Slpm to fuelBottleTestFlowRateN2Slpm and nitrogenIgnitionFlowRateN2Slpm to nitrogenTestFlowRateN2Slpm after stabilizationTime1Sec
constantFlowTimeSec:45,
time (in seconds) to wait at a constant flow before continuing script

---

numberOfRampHolds:0,
number of times gas flow will ramp and hold following constantFlowTimeSec
rampSizeFuelSlpm:0,
fuel bottle flow rate (SLPM N2 basis) to add to (or subtract from) the current fuel flow rate for each ramp
rampSizeN2Slpm:0,
nitrogen flow rate (SLPM N2 basis) to add to (or subtract from) the current fuel flow rate for each ramp
rampDurationSec:0,
time (in seconds) to ramp to next flow rate

holdDurationSec:0,
time (in seconds) to hold at the flow rate after each ramp

burnerTemperatureLimitC:250,
temperature limit (in degrees C) that ends the test point (stops all gas flow and performs post-flame camera configurations, data and imaging collection, and then closes the test point nicely)

burnerTemperatureLimitDurationSecs:3
time (in seconds) the burner thermocouple must be above burnerTemperatureLimitC before action taken to end the script

3. s-Flame Script

For a deeper understanding of the experimental procedure, the JavaScript used to control the s-Flame tests is presented here. This is the final version, used in the second round of testing, which incorporates a few minor changes that were made during operations:

2020-01-14 Added ignition detection prior to igniter arm retraction
2020-03-11 burnerOverLimitSec variable changed to global scope
2020-03-13 Fixed ramp-holds not ramping to desired flow delta

The first change was implemented near the end of the first round of testing, while the other two changes were made between the two rounds.

Note that the early portion of the script, beginning near the bottom of this page, includes an example set of parameter values which are overwritten by values specified for each test.

```
/**
 * SFlameTest.js
 * Script which executes an s-Flame test point.
 *
 * Assumptions:
 *    - Cameras are configured except that data camera exposure and analog camera shutter
 *      are set in this script, and the intensified camera gate and gain are set here.
 *    - ACME Camera lens and filter is configured. Intensified Camera filter is configured.
 *    - IPSUs being used, as specified in SystemDefineSetup command, are ready
 *    - Mass flow controllers are enabled and zeroed
 *    - PMTs are enabled
 *    - ExtinctionDetectionConfigure command has been issued and
ExtinctinctionDetectionSetState(Disabled)
 *    - Ignition detection parameters are set as desired using the IgnitionDetectionConfigure command.
 */

//Hook up to the runScript signal to show where to start execution
AcmeAvionicsController.runScript.connect(run);

//This variable must be named parameters so the ground and flight software can manipulate the values.
var parameters = {
   testpointName:"NONE",

   // data only recording
```

```
preIgnitionDataTimeSec:0,
postFlameDataTimeSec:15,

// IPSU imaging
useIPSUs:true,
ipsuTimeoutSec:40,
useAcmeCamera:true,
useIntensifiedCamera:true,
ipsu1PreIgnitionArchiveRate:100,
ipsu2PreIgnitionArchiveRate:100,
ipsu1TestArchiveRate:2,
ipsu2TestArchiveRate:2,
dataCameraPreIgnitionExposureUs:100000,
dataCameraIgnitionExposureUs:100000,
dataCameraTestExposureUs:650000,
dataCameraExtinctionExposureUs:650000,
analogCameraPreIgnitionShutter:2,
analogCameraIgnitionShutter:2,
analogCameraTestShutter:2,
analogCameraExtinctionShutter:0,
intensifierGainAmplitude:50000,  // not raw, 2500 - 80,000
preIgnitionImageTimeSec:0,
postFlameImagingTimeSec:3,

// exposure bracketing
useBracketedExposures:false,
bracketingImagesPerExposure:1,
bracketingNumberOfExposures:4,
bracketingExposureStartUsec:5000,
bracketingExposureMultiplierTimes100:1000,  // eg. 1050 = factor of 10.50

// ignition
igniterIgnitionPosition:0,
igniterPreheatPowerV:0,
igniterPreheatTimeMs:0,
igniterIgnitionPowerV:3.75,
igniterIgnitionDurationMs:2000,
igniterDelayBeforeRetractMs:1500,        // after the start of the ignition pulse
igniterRetractVelocityCountsPerSec:2000,  // velocity of retract if ignition is not detected

// ignition detection to power off igniter and send home with velocity in AcmeConfig.ini
useIgnitionDetection:true,

// extinction detection
extinctionDetectionStopsTest:true,

// PMTs
pmt1Gain:6.25,
```

```
  pmt2Gain:6.5,
  pmt3Gain:6.5,

  // fiber movement
  fiberIgnitionPosition:0,    // >0 to decrease time to get to test position
  fiberTestPosition:0,     // where to start back and forth movement
  fiberMoveDuringTest:false,
  fiberSegmentDistance:100,   // first move is toward the burner this amount, then back
  fiberVelocity:3000,
  fiberAcceleration:1000,
  fiberDeceleration:0,
  fiberHoldDurationSec:1,
  fiberCycles:100,          // can be large.  stopped when test is complete

  // gas flow
  useFoma:true,
  flowTimeoutSec: 120,
  nitrogenPriming1DurationSec:0,
  nitrogenPriming2DurationSec:0,
  fuelFlowIgnitionDeltaMs:1000,  // positive for fuel flow starting before ignition, negative for after
  stabilizationTime1Sec:4,
  stabilizationTime2Sec:4,
  fuelBottleIgnitionFlowRateN2Slpm:0.1,
  nitrogenIgnitionFlowRateN2Slpm:0,
  fuelBottleTestFlowRateN2Slpm:0.1,
  nitrogenTestFlowRateN2Slpm:0,
  ignitionToTestRampTimeSec:5,
  constantFlowTimeSec:30,

  numberOfRampHolds:0,
  rampSizeFuelSlpm:0, // each ramp
  rampSizeN2Slpm:0,   // each ramp
  rampDurationSec:2, // make 0 for steps
  holdDurationSec:10, // each step

  burnerTemperatureLimitC:250,
  burnerTemperatureLimitDurationSecs:3   // duration temp must be above limit before triggering
  };


// Print out the parameters so the ground software can display and support changes
printParams();

// variable for checkBurnerTemperature function
var burnerOverLimitSecs;


// Execution begins here when the RunScript command is issued
```

```
function run()
{
   // tell ACME telemetry that the script is running
   setScriptStateRunning();

   // print parameters for a record of what was used after any changes
   printParams();

   var cirManifoldsOpen = false;
   var ipsusStarted = false;
         burnerOverLimitSecs = 0;

   try
   {
      print("s-Flame test point " + parameters.testpointName + " STARTED");

      // Check that MFCS are in operate state
      if ( FlowTelemetry.MFCA2BypassState != 4) { print("WARNING: MFCA2 not in Operate state"); }
      if ( FlowTelemetry.MFCA3FuelState != 4) { print("WARNING: MFCA3 not in Operate state"); }

      // check that all of the flows are within the MFCs full scales. Throw exception if not.
      if ( parameters.nitrogenIgnitionFlowRateN2Slpm > FlowTelemetry.MFCA2BypassFullScaleFlowSlpm
)
         throw("INVALID PARAMETER: nitrogenIgnitionFlowRateN2Slpm exceeds MFCA2 maximum");
      if ( parameters.nitrogenTestFlowRateN2Slpm > FlowTelemetry.MFCA2BypassFullScaleFlowSlpm )
         throw("INVALID PARAMETER: nitrogenTestFlowRateN2Slpm exceeds MFCA2 maximum");
      if ( parameters.fuelBottleIgnitionFlowRateN2Slpm > FlowTelemetry.MFCA3FuelFullScaleFlowSlpm )
         throw("INVALID PARAMETER: fuelBottleIgnitionFlowRateN2Slpm exceeds MFCA3 maximum");
      if ( parameters.fuelBottleTestFlowRateN2Slpm > FlowTelemetry.MFCA3FuelFullScaleFlowSlpm )
         throw("INVALID PARAMETER: fuelBottleTestFlowRateN2Slpm exceeds MFCA3 maximum");

      if ( ExtinctionDetectionTelemetry.extinctionDetectionAveragingDuration <= 0 )
         throw ("Extinction detection averaging duration must be greater than zero.");

      // check that these parameters are multiples of 50 which is required in the ignition sequence
      // which loops every 50 msecs and uses exact msec timestamps
      if ( parameters.fuelFlowIgnitionDeltaMs % 50 > 0)
         throw("INVALID PARAMETER: fuelFlowIgnitionDeltaMs must be 0 or a multiple of 50");
      if ( parameters.igniterPreheatTimeMs % 50 > 0)
         throw("INVALID PARAMETER: igniterPreheatTimeMs must be 0 or a multiple of 50");
      if ( parameters.igniterDelayBeforeRetractMs % 50 > 0)
         throw("INVALID PARAMETER: igniterDelayBeforeRetractMs must be 0 or a multiple of 50");

      if ( DiagnosticTelemetry.PMTPower < 4.0) { print("WARNING: PMT's are not powered."); }
      print("Setting PMT gains");
      PmtSetAllGains.PMT1Gain = parameters.pmt1Gain;
      PmtSetAllGains.PMT2Gain = parameters.pmt2Gain;
      PmtSetAllGains.PMT3Gain = parameters.pmt3Gain;
```

```
    sendCommand(PmtSetAllGains);

    // Figure out and display the timing of ignition-related events
    // times are milliseconds from the start of N2 priming, even if that is not performed
    var fuelOpenValvesTimeMs = parameters.nitrogenPriming2DurationSec * 1000;
    var fuelOpenMfcsTimeMs = fuelOpenValvesTimeMs + 500;
    var igniterIgnitionTimeMs = fuelOpenValvesTimeMs + parameters.fuelFlowIgnitionDeltaMs;
    var igniterPreheatTimeMs = igniterIgnitionTimeMs - parameters.igniterPreheatTimeMs;
    var igniterRetractTimeMs = igniterIgnitionTimeMs + parameters.igniterDelayBeforeRetractMs;

    var minTime = Math.min(fuelOpenValvesTimeMs, fuelOpenMfcsTimeMs, igniterIgnitionTimeMs,
igniterPreheatTimeMs, igniterRetractTimeMs );
    if ( minTime < 0 )
    {
      // add to get every time above 0
      fuelOpenValvesTimeMs += (-1 * minTime);
      fuelOpenMfcsTimeMs += (-1 * minTime);
      igniterIgnitionTimeMs += (-1 * minTime);
      igniterPreheatTimeMs += (-1 * minTime);
      igniterRetractTimeMs += (-1 * minTime);
    }
    var stabilizationTime1Ms = Math.max(igniterRetractTimeMs,fuelOpenValvesTimeMs);

    print("Ignition Timing Values:");
    print("----------------------");
    if ( parameters.nitrogenIgnitionFlowRateN2Slpm > 0)
      print("Start of N2 priming (period 2): 0 Ms");
    print("Start of Fuel flow at ignition rate " + fuelOpenValvesTimeMs + " Ms");
    if ( parameters.igniterPreheatTimeMs > 0 )
      print("Start of igniter preheat: " + igniterPreheatTimeMs + " Ms");
    print("Start of ignition: " + igniterIgnitionTimeMs + " Ms");
    print("Start of igniter retraction: " + igniterRetractTimeMs + " Ms");
    print("Start of stabilization time 1: " + stabilizationTime1Ms + " Ms");
    print("----------------------");

    var burnerTemperatureLimitV =
convertKTypeThermocoupleTempToVolts(parameters.burnerTemperatureLimitC,
DiagnosticTelemetry.tcCj);
    print("temperature limit of " + parameters.burnerTemperatureLimitC + " C is " +
burnerTemperatureLimitV*1000 + "mV");

    // Open the test point
    if(parameters.testpointName == "NONE") { throw "Must define a test point name."; }
    if(parameters.testpointName.length > 7) { throw "Test point name must be 7 characters or less." }
    SystemOpenTestPoint.testpointName = parameters.testpointName;
    sendCommand(SystemOpenTestPoint);

    var usingNitrogen = (parameters.nitrogenIgnitionFlowRateN2Slpm>0)
```

```
    || (parameters.nitrogenTestFlowRateN2Slpm>0);
  var usingFuel = (parameters.fuelBottleIgnitionFlowRateN2Slpm>0)
    || (parameters.fuelBottleTestFlowRateN2Slpm>0);

  // Set camera settings
  print("Changing cameras to pre-ignition settings")
  if ( parameters.useIPSUs == true)
  {
    DataCameraSetManualExposure.integrationTimeUs =
parameters.dataCameraPreIgnitionExposureUs;
    sendCommand(DataCameraSetManualExposure);
  }
  AnalogCameraSetShutter.shutter = parameters.analogCameraPreIgnitionShutter;
  sendCommand(AnalogCameraSetShutter);

  // Put intensified camera in safe mode before turning on lights
  // gateType = 0 is pulsed mode
  print("Intensified camera to safe mode");
  IntensifiedCameraGate.gateType = 0;
  IntensifiedCameraGate.pulsedGateWidthCounts = 0;
  sendCommand(IntensifiedCameraGate);
  mSleep(500);
  IntensifiedCameraGainAmplitude.gainAmplitudeCounts = 2048;
  sendCommand(IntensifiedCameraGainAmplitude);
  // delay to get to safed mode?
  mSleep(2000);

  // Turn on chamber lights for viewing igniter insertion
  IlluminationConfigure.sideLEDBankState = 1;
  IlluminationConfigure.endLEDBankState = 1;
  sendCommand(IlluminationConfigure);

  // Insert the fiber
  if( SystemDefineSetup.efieldConfig == 0 )
  {
    print("Moving fiber to its starting position");
    FiberMove.position = parameters.fiberIgnitionPosition;
    FiberMove.velocity = 3000;
    sendCommand(FiberMove);
  }

  // Insert the igniter
  if( parameters.igniterIgnitionPosition != 0)
  {
    print("Inserting igniter");
    IgniterMove.position = parameters.igniterIgnitionPosition;
    sendCommand(IgniterMove);
  }
```

```
    // Disable extinction detection
    ExtinctionDetectionSetState.state = 0;
    sendCommand(ExtinctionDetectionSetState);

    if ( parameters.useFoma )
    {
      // clear and report initial MFC gas usage
      clearMfcIntegratedFlow();
      reportMfcIntegratedFlow(true);
    }

    // Pause here to check the igniter position and CIR manifold states
    print("Test point PAUSED to check igniter. SystemResumeScript when ready.");
    pauseScript();

    // Set the igniter ready to retract after ignition
    IgniterSetupMoveWithParameters.position = 0;
    IgniterSetupMoveWithParameters.velocity = parameters.igniterRetractVelocityCountsPerSec;
    IgniterSetupMoveWithParameters.acceleration = 5000;
    IgniterSetupMoveWithParameters.deceleration = 5000;
    sendCommand(IgniterSetupMoveWithParameters);

    // make sure ACME valves are closed before opening CIR manifolds
    sendCommand(FlowCloseAll);

    if ( parameters.useFoma && (usingNitrogen))
    {
      print("Opening CIR nitrogen manifold");
      // This function in helpers.js waits until complete or throws an error.
      // The parameters are (N2, O2, Diluent, Fuel, N2 Bypass, delayMs)
      openManifolds(true, false, false, false, true, 6000);

      cirManifoldsOpen = true;
    }
    var cirManifoldStartTime = new Date().getTime();

    // priming N2 period 1
    if ( (parameters.nitrogenIgnitionFlowRateN2Slpm > 0) &&
(parameters.nitrogenPriming1DurationSec > 0))
    {
      // open N2 valve and MFC and prime the nitrogen
      FlowSetSolenoidValveState.solenoidValve = 7;
      FlowSetSolenoidValveState.state = 1;
      sendCommand(FlowSetSolenoidValveState);

      mSleep(500);
```

```
    FlowSetMfcRate.massFlowController = 2;
    FlowSetMfcRate.flowSlm = parameters.nitrogenIgnitionFlowRateN2Slpm;
    sendCommand(FlowSetMfcRate);

    // for TADS timestamp of MFCs open
    print("Priming gas lines for");

    print("Priming nitrogen (period 1) for " + parameters.nitrogenPriming1DurationSec + " sec");
    var startTime = new Date().getTime();
    while ( updateDurationRemaining(startTime, parameters.nitrogenPriming1DurationSec) > 0 )
    {
       mSleep(1000);
    }
}

// Start pre-ignition data capture and wait the prescribed time.
print("Recording pre-ignition data for " + parameters.preIgnitionDataTimeSec + " sec");
sendCommand(DataRecordingStart);
var secsRemaining = parameters.preIgnitionDataTimeSec;
SystemTelemetry.scriptDurationRemaining = secsRemaining;
while ( secsRemaining > 0 )
{
    mSleep(1000);
    secsRemaining--;
    SystemTelemetry.scriptDurationRemaining = secsRemaining;
}

// Start the IPSU(s) recording images
if ( parameters.useIPSUs == true)
{
    print("Setting IPSU(s) pre-ignition archive rates");
    // set the pre-ignition archive rate for IPSU1
    IpsuSetArchiveRate.ipsu = 1;
    IpsuSetArchiveRate.archiveRate = parameters.ipsu1PreIgnitionArchiveRate;
    sendCommand(IpsuSetArchiveRate);

    // need delay between identical commands so 2nd set of params not used in 1st command
    mSleep(100);

    // set the pre-ignition archive rate for IPSU2
    IpsuSetArchiveRate.ipsu = 2;
    IpsuSetArchiveRate.archiveRate = parameters.ipsu2PreIgnitionArchiveRate;
    sendCommand(IpsuSetArchiveRate);

    // This is a flag that gets set by the ImagingController if there is trouble
    // talking to an IPSU
    ImagingController.ipsuErrorFlag = false;
```

```
    sendStartIpsusCommand();

    print("Starting IPSU(s) archiving.  Waiting up to " + parameters.ipsuTimeoutSec + " sec");
    var secsRemaining = parameters.ipsuTimeoutSec;
    while ( secsRemaining > 0 )
    {
      SystemTelemetry.scriptDurationRemaining = secsRemaining;
      if ( IpsuTelemetry.ipsuPhase == 5 ) { break; }
      if ( ImagingController.ipsuErrorFlag == true ) { break; }
      mSleep(1000);
      secsRemaining--;
    }

    if ( ImagingController.ipsuErrorFlag == true ) {  throw "IPSU error" };
    if ( secsRemaining <= 0 ) {  throw "IPSU timeout error" };
    ipsusStarted = true;

    print("Recording pre-ignition images for " + parameters.preIgnitionImageTimeSec + " sec");
    secsRemaining = parameters.preIgnitionImageTimeSec;
    SystemTelemetry.scriptDurationRemaining = secsRemaining;
    while ( secsRemaining > 0 )
    {
      mSleep(1000);
      secsRemaining--;
      SystemTelemetry.scriptDurationRemaining = secsRemaining;
    }
  }

  if ( parameters.useFoma && usingFuel)
  {
    // under certain conditions a delay is needed between CIR scripts
    if ( cirManifoldsOpen == true &&  ( new Date().getTime() - cirManifoldStartTime < 6000 ))
    {
      print("Waiting 6 seconds between CIR manifold scripts");
      // CIR open manifold script was run and 6 seconds have not yet elapsed
      mSleep(6000);
    }

    print("Opening CIR fuel manifold");
    openFuelManifold();
    cirManifoldsOpen = true;
  }

  // Turn off the lights
  IlluminationConfigure.sideLEDBankState = 0;
  IlluminationConfigure.endLEDBankState = 0;
  sendCommand(IlluminationConfigure);
```

```
    // Put intensified camera in continuous mode and set gain
    // gateType = 1 is continuous mode.  pulsed gate width parameter is ignored.
    print("Intensified camera to continuous mode");
    IntensifiedCameraGate.gateType = 1;
    IntensifiedCameraGate.pulsedGateWidthCounts = 0;
    sendCommand(IntensifiedCameraGate);
    mSleep(500);
    IntensifiedCameraGainAmplitude.gainAmplitudeCounts =
intensifierGainToRaw(parameters.intensifierGainAmplitude);
    sendCommand(IntensifiedCameraGainAmplitude);

    // Change camera settings
    print("Changing cameras to ignition settings")
    if ( parameters.useIPSUs )
    {
      if ( parameters.useBracketedExposures )
      {
        print("Starting bracketed exposures on ACME camera");
        DataCameraExposureBracketing.enable = true;
        DataCameraExposureBracketing.imagesPerExposure =
parameters.bracketingImagesPerExposure;
        DataCameraExposureBracketing.exposures = parameters.bracketingNumberOfExposures;
        DataCameraExposureBracketing.exposureStartUsec =
parameters.bracketingExposureStartUsec;
        DataCameraExposureBracketing.exposureIncrementUsec =
parameters.bracketingExposureMultiplierTimes100;
        sendCommand(DataCameraExposureBracketing);
      }
      else
      {
        DataCameraExposureBracketing.enable = false;
        sendCommand(DataCameraExposureBracketing);

        DataCameraSetManualExposure.integrationTimeUs =
parameters.dataCameraIgnitionExposureUs;
        sendCommand(DataCameraSetManualExposure);
      }
    }

    AnalogCameraSetShutter.shutter = parameters.analogCameraIgnitionShutter;
    sendCommand(AnalogCameraSetShutter);

    // priming N2 period 2
    if ( (parameters.nitrogenIgnitionFlowRateN2Slpm > 0) &&
(parameters.nitrogenPriming2DurationSec > 0))
    {
      // open N2 valve and MFC and prime the nitrogen
      FlowSetSolenoidValveState.solenoidValve = 7;
```

```
    FlowSetSolenoidValveState.state = 1;
    sendCommand(FlowSetSolenoidValveState);

    mSleep(500);

    FlowSetMfcRate.massFlowController = 2;
    FlowSetMfcRate.flowSlm = parameters.nitrogenIgnitionFlowRateN2Slpm;
    sendCommand(FlowSetMfcRate);

    if ( parameters.nitrogenPriming1DurationSec == 0)
    {
       // for TADS timestamp of MFCs open
       print("Priming gas lines for");
    }

    print("Priming nitrogen (period 2) for " + parameters.nitrogenPriming2DurationSec + " sec");
    // priming switches over to mixed flow at the appropriate time in the timing loop below
  }

  if ( parameters.useIPSUs )
  {
    print("Changing IPSU(s) to test archive rate");
    // set the test archive rate for IPSU1
    IpsuSetArchiveRate.ipsu = 1;
    IpsuSetArchiveRate.archiveRate = parameters.ipsu1TestArchiveRate;
    sendCommand(IpsuSetArchiveRate);

    // need delay between identical commands so 2nd set of params not used in 1st command
    mSleep(100);

    // set the test archive rate for IPSU2
    IpsuSetArchiveRate.ipsu = 2;
    IpsuSetArchiveRate.archiveRate = parameters.ipsu2TestArchiveRate;
    sendCommand(IpsuSetArchiveRate);
  }

  // start ignition detection
  if ( parameters.useIgnitionDetection)
  {
    IgnitionDetectionSetState.state = 1;
    sendCommand(IgnitionDetectionSetState);
  }

  var flowStartTime = new Date().getTime();
  var timedOut = false;

  // iterate through the timed ignition events
```

```
    var maxTime = Math.max(fuelOpenValvesTimeMs, fuelOpenMfcsTimeMs, igniterIgnitionTimeMs,
igniterPreheatTimeMs, igniterRetractTimeMs );
    var timeStampMs = 0;
    while ( timeStampMs <= maxTime )
    {
      if ( timeStampMs == fuelOpenValvesTimeMs)
      {
        // Start fuel flow (and N2 flow if not already priming)
        print("Starting fuel flow at ignition flow rate");
        // Open valves
        FlowSetAllSolenoidValveStates.SVA1State = 0;
        FlowSetAllSolenoidValveStates.SVA2State = 0;
        FlowSetAllSolenoidValveStates.SVA3State = 0;
        FlowSetAllSolenoidValveStates.SVA4State = 0;
        FlowSetAllSolenoidValveStates.SVA5State = 0;
        FlowSetAllSolenoidValveStates.SVA6State = 0;
        // N2 to burner
        FlowSetAllSolenoidValveStates.SVA7State = usingNitrogen ? 1 : 0;
        FlowSetAllSolenoidValveStates.SVA8State = 0;
        // Fuel to burner
        FlowSetAllSolenoidValveStates.SVA9State = usingFuel ? 1 : 0;
        sendCommand(FlowSetAllSolenoidValveStates);
      }

      if ( timeStampMs == fuelOpenMfcsTimeMs )
      {
        // Set the MFCs to ignition flow rates
        // separate from valve opening because want a delay between
        FlowSetAllMfcRates.MFCA1StaticMixerFlowSlm = 0;
        FlowSetAllMfcRates.MFCA2BypassFlowSlm = parameters.nitrogenIgnitionFlowRateN2Slpm;
        FlowSetAllMfcRates.MFCA3FuelFlowSlm = parameters.fuelBottleIgnitionFlowRateN2Slpm;
        sendCommand(FlowSetAllMfcRates);

        // Mark start time for flow timeout.
        print("Flow timeout in " + parameters.flowTimeoutSec + " sec");
        flowStartTime = new Date().getTime();
      }

      if ( timeStampMs == igniterPreheatTimeMs)
      {
        // preheat igniter
        if (( parameters.igniterPreheatTimeMs > 0) && (parameters.igniterPreheatPowerV > 0))
        {
          var preheatDurationMs = parameters.igniterPreheatTimeMs;
          if ( parameters.igniterIgnitionDurationMs > 0 )
          {
            // add to the duration so there is overlap with ignition and no drop out
            preheatDurationMs += 500;
```

```
            }
            print("Preheating igniter with " + parameters.igniterPreheatPowerV + "V for " +
parameters.igniterPreheatTimeMs + " Ms");
            IgniterPulse.power = parameters.igniterPreheatPowerV;
            IgniterPulse.durationMs = preheatDurationMs;
            sendCommand(IgniterPulse);
          }
        }

        if ( timeStampMs == igniterIgnitionTimeMs )
        {
          // ignition
          if ( (parameters.igniterIgnitionPowerV > 0) && (parameters.igniterIgnitionDurationMs > 0))
          {
            print("Igniting with " + parameters.igniterIgnitionPowerV + "V for " +
parameters.igniterIgnitionDurationMs + " Ms");
            IgniterPulse.power      = parameters.igniterIgnitionPowerV;
            IgniterPulse.durationMs = parameters.igniterIgnitionDurationMs;
            sendCommand(IgniterPulse);
          }
        }

        if ( timeStampMs == igniterRetractTimeMs )
        {
          if ( parameters.useIgnitionDetection)
          {
            // disable ignition detection so it doesn't interrupt igniter retraction
            IgnitionDetectionSetState.state = 0;
            sendCommand(IgnitionDetectionSetState);
          }

          // retract igniter
          print("Retracting igniter");
          sendCommand(IgniterExecuteSetupMove);
        }

        // increment time
        timeStampMs += 50;
        mSleep(50);
      }

      // Change camera settings
      print("Changing cameras to test settings")
      if ( parameters.useIPSUs )
      {
        if ( !parameters.useBracketedExposures )
        {
          DataCameraSetManualExposure.integrationTimeUs = parameters.dataCameraTestExposureUs;
```

```
            sendCommand(DataCameraSetManualExposure);
        }
    }

    AnalogCameraSetShutter.shutter = parameters.analogCameraTestShutter;
    sendCommand(AnalogCameraSetShutter);

    // waiting stabilization time 1
    print("Waiting stabilization time #1 " + parameters.stabilizationTime1Sec + " sec");
    var startTime = new Date().getTime();
    while ( updateDurationRemaining(startTime, parameters.stabilizationTime1Sec) > 0 )
    {
        checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
parameters.extinctionDetectionStopsTest );
        checkBurnerTemperature(burnerTemperatureLimitV);
        mSleep(1000);
    }

    // Enable extinction detection
    ExtinctionDetectionSetState.state = 1;
    sendCommand(ExtinctionDetectionSetState);
    print("Extinction detection enabled");

    // Ramp gas flow from ignition rates to test rates
    // A ramp rate of zero in the FlowRamped command leaves that MFC at its last setting
    if ( (parameters.nitrogenTestFlowRateN2Slpm != parameters.nitrogenIgnitionFlowRateN2Slpm)
        || (parameters.fuelBottleTestFlowRateN2Slpm !=
parameters.fuelBottleIgnitionFlowRateN2Slpm))
    {
        if ( parameters.ignitionToTestRampTimeSec > 0 )
            FlowRamped.rampDurationSec = parameters.ignitionToTestRampTimeSec;
        else
            FlowRamped.rampDurationSec = 0.1;
        FlowRamped.holdDurationSec = 0;

        FlowRamped.MFCA1MinimumFlowSlm = 0;
        FlowRamped.MFCA1MaximumFlowSlm = 0;
        FlowRamped.MFCA1RampRateSlmPerSec = 0;

        FlowRamped.MFCA2MinimumFlowSlm = ( parameters.nitrogenTestFlowRateN2Slpm >
parameters.nitrogenIgnitionFlowRateN2Slpm ) ? parameters.nitrogenIgnitionFlowRateN2Slpm :
parameters.nitrogenTestFlowRateN2Slpm;
        FlowRamped.MFCA2MaximumFlowSlm = ( parameters.nitrogenTestFlowRateN2Slpm >
parameters.nitrogenIgnitionFlowRateN2Slpm ) ? parameters.nitrogenTestFlowRateN2Slpm :
parameters.nitrogenIgnitionFlowRateN2Slpm;
        FlowRamped.MFCA2RampRateSlmPerSec = (parameters.nitrogenTestFlowRateN2Slpm -
parameters.nitrogenIgnitionFlowRateN2Slpm) / FlowRamped.rampDurationSec;
```

```
        FlowRamped.MFCA3MinimumFlowSlm = ( parameters.fuelBottleTestFlowRateN2Slpm >
parameters.fuelBottleIgnitionFlowRateN2Slpm ) ? parameters.fuelBottleIgnitionFlowRateN2Slpm :
parameters.fuelBottleTestFlowRateN2Slpm;
        FlowRamped.MFCA3MaximumFlowSlm = ( parameters.fuelBottleTestFlowRateN2Slpm >
parameters.fuelBottleIgnitionFlowRateN2Slpm ) ? parameters.fuelBottleTestFlowRateN2Slpm :
parameters.fuelBottleIgnitionFlowRateN2Slpm;
        FlowRamped.MFCA3RampRateSlmPerSec = (parameters.fuelBottleTestFlowRateN2Slpm -
parameters.fuelBottleIgnitionFlowRateN2Slpm) / FlowRamped.rampDurationSec;

        print("Ramping gas flow from ignition to test rates in " + parameters.ignitionToTestRampTimeSec
+ " secs");
        sendCommand(FlowRamped);
      }

      if ( parameters.ignitionToTestRampTimeSec > 0 )
      {
        var startTime = new Date().getTime();
        while ( updateDurationRemaining(startTime, parameters.ignitionToTestRampTimeSec) > 0 )
        {
          checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
                    parameters.extinctionDetectionStopsTest );
          checkBurnerTemperature(burnerTemperatureLimitV);
          mSleep(1000);
        }

        // make sure ramp is done
        while (FlowTelemetry.MFCA3FuelWaveform != 0)
        {
          checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
                      parameters.extinctionDetectionStopsTest );
          checkBurnerTemperature(burnerTemperatureLimitV);
          mSleep(1000);
        }
      }

      // waiting stabilization time 2
      print("Waiting stabilization time #2 " + parameters.stabilizationTime2Sec + " sec");
      var startTime = new Date().getTime();
      while ( updateDurationRemaining(startTime, parameters.stabilizationTime2Sec) > 0 )
      {
        checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
parameters.extinctionDetectionStopsTest );
        checkBurnerTemperature(burnerTemperatureLimitV);
        mSleep(1000);
      }

      // Start moving fiber to test position
      if ( parameters.fiberTestPosition > 0 && SystemDefineSetup.efieldConfig == 0 )
```

```
    {
      print("Moving fiber to test position");
      FiberMove.position = parameters.fiberTestPosition;
      FiberMove.velocity = 3000;
      sendCommand(FiberMove);
      mSleep(2000);
    }

    // start fiber back and forth
    if ( parameters.fiberMoveDuringTest && SystemDefineSetup.efieldConfig == 0 )
    {
      // wait until fiber is stopped
      print("Waiting for fiber to reach test position");
      fiberLastPosition = -1;
      while ( fiberLastPosition != FiberTelemetry.fiberPosition)
      {
        fiberLastPosition = FiberTelemetry.fiberPosition;
        mSleep(2000);

        checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
parameters.extinctionDetectionStopsTest );
        checkBurnerTemperature(burnerTemperatureLimitV);

      }
      print("Starting fiber movement");
      // Moves from test position toward burner first, then back
      FiberRampedWaveMove.segmentDistanceA = parameters.fiberSegmentDistance;
      FiberRampedWaveMove.segmentDistanceB = -1 * parameters.fiberSegmentDistance;
      FiberRampedWaveMove.velocityA = parameters.fiberVelocity;
      FiberRampedWaveMove.velocityB = parameters.fiberVelocity;
      FiberRampedWaveMove.accelerationA = parameters.fiberAcceleration;
      FiberRampedWaveMove.accelerationB = parameters.fiberAcceleration;
      FiberRampedWaveMove.decelerationA = parameters.fiberDeceleration;
      FiberRampedWaveMove.decelerationB = parameters.fiberDeceleration;
      FiberRampedWaveMove.holdDurationA = parameters.fiberHoldDurationSec;
      FiberRampedWaveMove.holdDurationB = parameters.fiberHoldDurationSec;
      FiberRampedWaveMove.cycles = parameters.fiberCycles;
      sendCommand(FiberRampedWaveMove);
    }

    print("Waiting constant flow time " + parameters.constantFlowTimeSec + " sec");
    if ( parameters.constantFlowTimeSec > 0 )
    {
      var startTime = new Date().getTime();
      while ( updateDurationRemaining(startTime, parameters.constantFlowTimeSec) > 0 )
      {
        checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
                        parameters.extinctionDetectionStopsTest );
```

```
            checkBurnerTemperature(burnerTemperatureLimitV);
            mSleep(1000);
          }
        }

        if ( parameters.numberOfRampHolds > 0)
        {
          // perform the ramp holds
          if ( parameters.rampDurationSec == 0)
          { FlowRamped.rampDurationSec = 0.1; }
          else { FlowRamped.rampDurationSec = parameters.rampDurationSec; }

          FlowRamped.holdDurationSec = parameters.holdDurationSec;

          FlowRamped.MFCA1MinimumFlowSlm = 0;
          FlowRamped.MFCA1MaximumFlowSlm = 0;
          FlowRamped.MFCA1RampRateSlmPerSec = 0;

          FlowRamped.MFCA2MinimumFlowSlm = ( parameters.rampSizeN2Slpm >= 0 ) ?
parameters.nitrogenTestFlowRateN2Slpm : parameters.nitrogenTestFlowRateN2Slpm +
parameters.numberOfRampHolds * parameters.rampSizeN2Slpm - 1e-6;
          FlowRamped.MFCA2MaximumFlowSlm = ( parameters.rampSizeN2Slpm >= 0 ) ?
parameters.nitrogenTestFlowRateN2Slpm + parameters.numberOfRampHolds *
parameters.rampSizeN2Slpm + 1e-6 : parameters.nitrogenTestFlowRateN2Slpm;
          FlowRamped.MFCA2RampRateSlmPerSec = parameters.rampSizeN2Slpm /
FlowRamped.rampDurationSec;

          FlowRamped.MFCA3MinimumFlowSlm = ( parameters.rampSizeFuelSlpm >= 0 ) ?
parameters.fuelBottleTestFlowRateN2Slpm : parameters.fuelBottleTestFlowRateN2Slpm +
parameters.numberOfRampHolds * parameters.rampSizeFuelSlpm - 1e-6;
          FlowRamped.MFCA3MaximumFlowSlm = ( parameters.rampSizeFuelSlpm >= 0 ) ?
parameters.fuelBottleTestFlowRateN2Slpm + parameters.numberOfRampHolds *
parameters.rampSizeFuelSlpm + 1e-6: parameters.fuelBottleTestFlowRateN2Slpm;
          FlowRamped.MFCA3RampRateSlmPerSec = parameters.rampSizeFuelSlpm /
FlowRamped.rampDurationSec;

          print("Starting ramp holds");
          sendCommand(FlowRamped);

          if (( parameters.numberOfRampHolds * (parameters.rampDurationSec +
parameters.holdDurationSec) - parameters.holdDurationSec) > 2)
          {
            // Wait for flow waveform to go back to None
            var startTime = new Date().getTime();
            secsRemaining = parameters.flowTimeoutSec - Math.floor((startTime - flowStartTime) / 1000);
            print("Waiting for end of ramp holds or flow timeout in " + secsRemaining + " sec");
            var waveformStarted = false;
            var waveformEnded = false;
```

```
            while ((updateDurationRemaining(startTime, secsRemaining) > 0) && (waveformEnded !=
true))
            {
               checkExtinctionDetected(flowStartTime, parameters.flowTimeoutSec,
                        parameters.extinctionDetectionStopsTest);
               checkBurnerTemperature(burnerTemperatureLimitV);
               mSleep(1000);

              // wait for waveform state to start then wait for it to end
              if ( waveformStarted == false )
              {
                 if ((FlowTelemetry.MFCA3FuelWaveform != 0 ) || (FlowTelemetry.MFCA2BypassWaveform
!= 0))
                 {  waveformStarted = true;  }
              }
              else if (( FlowTelemetry.MFCA3FuelWaveform == 0) &&
(FlowTelemetry.MFCA2BypassWaveform == 0))
                {  waveformEnded = true; }
            }
         }

         // ramped command doesn't include a final hold
         print("Waiting final hold duration " + parameters.holdDurationSec + " sec");
         if ( parameters.holdDurationSec > 0 )
         {
           var startTime = new Date().getTime();
           while ( updateDurationRemaining(startTime, parameters.holdDurationSec) > 0 )
           {
               checkExtinctionDetected( flowStartTime, parameters.flowTimeoutSec,
                           parameters.extinctionDetectionStopsTest );
               checkBurnerTemperature(burnerTemperatureLimitV);
               mSleep(1000);
           }
         }

      }

      // TEST_ENDING

      // Change camera settings
      print("Changing cameras to extinction settings")
      if ( parameters.useIPSUs )
      {
         if ( ! parameters.useBracketedExposures )
         {
            DataCameraSetManualExposure.integrationTimeUs =
parameters.dataCameraExtinctionExposureUs;
            sendCommand(DataCameraSetManualExposure);
```

```
    }
  }

  AnalogCameraSetShutter.shutter = parameters.analogCameraExtinctionShutter;
  sendCommand(AnalogCameraSetShutter);

  if ( parameters.useIgnitionDetection)
  {
    // disable ignition detection
    IgnitionDetectionSetState.state = 0;
    sendCommand(IgnitionDetectionSetState);
  }

  // Disable extinction detection
  ExtinctionDetectionSetState.state = 0;
  sendCommand(ExtinctionDetectionSetState);

  print("Turning off gas flow");
  sendCommand(FlowCloseAll);

  if ( parameters.useFoma )
  {
    // this does not wait for complete
    closeManifolds();
  }

  // Post-flame imaging
  if ( ipsusStarted )
  {
    print("Waiting post-flame imaging duration of " + parameters.postFlameImagingTimeSec + "
sec");
    var startTime = new Date().getTime();

    while ( updateDurationRemaining(startTime, parameters.postFlameImagingTimeSec) > 0 )
    {
      mSleep(1000);
    }

    print("Stopping IPSU(s)");
    sendCommand(StopIpsus);
  }

  sendCommand(FiberHaltMove);
  mSleep(2000);

  // Post-flame data
  print("Waiting post-flame data duration of " + parameters.postFlameDataTimeSec + " sec");
  var startTime = new Date().getTime();
```

```
    while ( updateDurationRemaining(startTime, parameters.postFlameDataTimeSec) > 0 )
    {
       mSleep(1000);
    }
    sendCommand(SystemCloseTestPoint);

    print("Homing fiber");
    FiberMove.position=0;
    FiberMove.velocity=3000;
    sendCommand(FiberMove);

    // Put intensified camera back in safe mode
    print("Intensified camera to safe mode");
    IntensifiedCameraGate.gateType = 0;
    IntensifiedCameraGate.pulsedGateWidthCounts = 0;
    sendCommand(IntensifiedCameraGate);
    mSleep(500);
    IntensifiedCameraGainAmplitude.gainAmplitudeCounts = 2048;
    sendCommand(IntensifiedCameraGainAmplitude);

    if ( parameters.useBracketedExposures )
    {
       // Turn off bracketed exposures
       DataCameraExposureBracketing.enable = false;
       sendCommand(DataCameraExposureBracketing);
    }

    print("Setting PMT gains to zero");
    PmtSetAllGains.PMT1Gain = 0;
    PmtSetAllGains.PMT2Gain = 0;
    PmtSetAllGains.PMT3Gain = 0;
    sendCommand(PmtSetAllGains);

    if ( parameters.useFoma )
    {
       // report final MFC gas usage
       reportMfcIntegratedFlow(false);
    }

    print("s-Flame test point " + parameters.testpointName + " COMPLETE");

    setScriptStateComplete();
 }
 catch(error)
 {
    print("s-Flame test point " + parameters.testpointName + " ABORTED.  Reason: " + error);

    setScriptStateAborted();
```

```
    print("Changing cameras to extinction settings")
    if ( ipsusStarted )
    {
      if ( ! parameters.useBracketedExposures )
      {
        DataCameraSetManualExposure.integrationTimeUs =
parameters.dataCameraExtinctionExposureUs;
        sendCommand(DataCameraSetManualExposure);
      }
    }

    AnalogCameraSetShutter.shutter = parameters.analogCameraExtinctionShutter;
    sendCommand(AnalogCameraSetShutter);

    if ( parameters.useIgnitionDetection)
    {
      // disable ignition detection
      IgnitionDetectionSetState.state = 0;
      sendCommand(IgnitionDetectionSetState);
    }

    // disable extinction detection and any action that may result
    ExtinctionDetectionSetState.state = 0;
    sendCommand(ExtinctionDetectionSetState);

    print("Turning off gas flow");
    // close ACME valves
    sendCommand(FlowCloseAll);

    sendCommand(FiberHaltMove);
    mSleep(1000);

    // close CIR valves
    if ( cirManifoldsOpen )
    {
      // this does not wait for complete
      closeManifolds();
    }

    if ((error == "EXTINCTION DETECTED") || (error == "Flow timeout reached"))
    {
       // Still do post-flame imaging
      if ( ipsusStarted )
      {
        print("Waiting post-flame imaging duration of " + parameters.postFlameImagingTimeSec + "
sec");
        var startTime = new Date().getTime();
```

```
      while ( updateDurationRemaining(startTime, parameters.postFlameImagingTimeSec) > 0 )
      {
         mSleep(1000);
      }

      print("Stopping IPSU(s)");
      sendCommand(StopIpsus);
   }

   // Still do post-flame data
   print("Waiting post-flame data duration of " + parameters.postFlameDataTimeSec + " sec");
   var startTime = new Date().getTime();

   while ( updateDurationRemaining(startTime, parameters.postFlameDataTimeSec) > 0 )
   {
      mSleep(1000);
   }
   sendCommand(SystemCloseTestPoint);
}
else
{
   print("Stopping data and image collection");
   // stop data collection
   sendCommand(SystemCloseTestPoint);

   // stop IPSUs from archiving images
   if ( ipsusStarted )
   {
      sendCommand(StopIpsus);
   }
}

// home the igniter
print("Homing igniter");
IgniterMove.position = 0;
sendCommand(IgniterMove);

// home the fiber
print("Homing fiber");
FiberMove.position=0;
FiberMove.velocity=3000;
sendCommand(FiberMove);

// Put intensified camera back in safe mode
print("Intensified camera to safe mode");
IntensifiedCameraGate.gateType = 0;
IntensifiedCameraGate.pulsedGateWidthCounts = 0;
```

```
      sendCommand(IntensifiedCameraGate);
      mSleep(500);
      IntensifiedCameraGainAmplitude.gainAmplitudeCounts = 2048;
      sendCommand(IntensifiedCameraGainAmplitude);

      if ( parameters.useBracketedExposures )
      {
         // Turn off bracketed exposures
         DataCameraExposureBracketing.enable = false;
         sendCommand(DataCameraExposureBracketing);
      }

      print("Setting PMT gains to zero");
      PmtSetAllGains.PMT1Gain = 0;
      PmtSetAllGains.PMT2Gain = 0;
      PmtSetAllGains.PMT3Gain = 0;
      sendCommand(PmtSetAllGains);

      if ( parameters.useFoma )
      {
         // report final MFC gas usage
         reportMfcIntegratedFlow(false);
      }

      print("s-Flame test point " + parameters.testpointName + " complete after abort");

   }

}

function checkExtinctionDetected(flowStartTime, timeoutSec, abortTest)
{
   // if abortTest is true this throws a message if extinction is detected
   // or if the timeout has been exceeded
   // if abortTest is false this returns 2 if extinction is detected, 0 if extinction
   // is disabled, 3 if a timeout has been exceeded, and 1 if none of these is true.
   var currentTime = new Date().getTime();

   var timedOut = false;
   var detected = false;
   var disabled = false;
   if((currentTime - flowStartTime) >= (timeoutSec * 1000))
   {
      timedOut = true;
   throw "Flow timeout reached";
   }

   if ( ExtinctionDetectionTelemetry.extinctionDetectionState == 2)
```

```
   {
      detected = true;
   }

   if ( ExtinctionDetectionTelemetry.extinctionDetectionState == 0)
   {
      disabled = true;
   }

   if (abortTest==true)
   {
      if (detected==true)
      { throw "EXTINCTION DETECTED"; }
   }

   if ( detected == true)
   { return 2;}
   else if ( timedOut == true )
   { return 3;}
   else if ( disabled == true )
   { return 0; }
   else
   { return 1; }

}


function updateDurationRemaining(startTime, startDurationSecs)
{
   var currentTime = new Date().getTime();
   elapsedSecs = Math.floor((currentTime - startTime) / 1000);
   SystemTelemetry.scriptDurationRemaining = startDurationSecs - elapsedSecs;
   return SystemTelemetry.scriptDurationRemaining;
}

function sendStartIpsusCommand()
{
   // Start the sequence of commands sent to IPSU(s) to open a test point
   // and start archiving images
   StartIpsus.testpointName = parameters.testpointName;
   StartIpsus.ipsu5NumberCameras = 0;
   if ( parameters.useAcmeCamera == true )
   {
      StartIpsus.ipsu5NumberCameras = 1;
      StartIpsus.ipsu5Camera1 = 3;
      if ( parameters.useIntensifiedCamera == true )
      {
         StartIpsus.ipsu5NumberCameras = 2;
```

```
        StartIpsus.ipsu5Camera2 = 4;
      }
   }
   else if ( parameters.useIntensifiedCamera == true )
   {
      StartIpsus.ipsu5NumberCameras = 1;
      StartIpsus.ipsu5Camera1 = 4;
   }
   sendCommand(StartIpsus);
}

function checkBurnerTemperature(temperatureLimitV)
{
   if ( DiagnosticTelemetry.tcBurner > temperatureLimitV)
   {
      burnerOverLimitSecs = burnerOverLimitSecs + 1;
      if ( burnerOverLimitSecs >= parameters.burnerTemperatureLimitDurationSecs )
      {
         throw "Burner temperature limit exceeded for specified duration";
      }
   }
   else
   {
      burnerOverLimitSecs = 0;
   }
}
```

4. Test Matrices

For all ACME experiments, including s-Flame, the test planning is refined in multiple steps as described in this section. Despite refinement, the experiments are always expected to address the science objectives approved in the ACME ISRD.

A. Test Matrices: Years in Advance

The test matrices found in section 5 of the ACME Integrated Science Requirements Document (ISRD, RDR version, Rev. B) were the basis for planning the equipment and resources, such as gas bottles, needed to carry out each of the NASA-selected ACME experiments. For s-Flame, the test matrices are found in section 5.4. The investigators were each directed to define matrices with required testing which included at most one burner configuration and no more than nominally 50 flames ignited. Regarding these constraints specified by Project Scientist Dennis Stocker, the intent was to conduct focused rather than comprehensive research given the considerable resources required for testing on the International Space Station. Meanwhile, there were no limits put on desired or highly desired tests because of NASA's plan to only carry out tests designated as required. However, in the case of s-Flame, the decision was made to also carry out their highly desired tests, i.e., treating them as required, so that its testing would be comparable to that of the other NASA-selected ACME experiments.

NASA Glenn's ISS Control Board later decided to limit ACME testing to nominally 4 months of ISS testing per experiment which with preparatory operations, minor delays, repeat testing due to science loss, etc. were expected to typically occur in a total of 5 months. This decision was made with the expectation that the required testing, e.g., as specified in the ISRD, could at least mostly be accomplished in that time period assuming the availability of crew and other needed resources.

As part of his planning, Stocker instigated an alternating sequence of experiment testing, wherein investigators would get a break in their testing in order to analyze early results in depth and with that knowledge achieve higher science outcomes in their second round of testing. As a result, each experiment was expected to get 2 months of testing per round, which meant nominally 8 test days per round of testing. Within that time constraint and available resources, experiments were ultimately permitted to plan for and conduct more tests than had been specified as required in the ISRD, despite the initial planning which ignored desired and highly desired testing.

B. Test Matrices: Months in Advance

At least a few months in advance of testing, ACME investigators are directed to submit a refined test matrix for the upcoming round corresponding to available hardware, gas bottles, and the agreed number of test days. The refinement can be based on what is learned from experimental, analytical, or computational efforts subsequent to the 2013 ISRD. Generally, this planning has been done in detail comparable to that found in the ISRD, with the important exception that it must furthermore establish the required hardware, including the specific mass flow controllers (and thus specification of the flow ranges) and gas bottles so that overall experiment operations, including crew activity, can be adequately planned. In this regard, it must establish general operational plans for the chamber atmosphere on each day of testing including filling, scrubbing (via circulation through the adsorber cartridge), venting (whether partial or complete), and measurements, if any, with the Gas Chromatograph Instrumentation Package (GCIP).

C. Test Matrices: Days in Advance

The investigators are required to submit their test parameters for all planned tests by noon Eastern at least three business days in advance of each scheduled test day. That submission includes all of the test parameters within the script and additional ones which are outside of the script Most of the latter deal with the camera settings which don't change during a test, for example the filters used with the ACME Data Camera (DataCam) and the Cool Flames Investigation (CFI) intensified camera. Outside of the script, the investigators must again specify the chamber atmosphere(s) and when GCIP measurements, if any, are planned where those must correspond to the plan submitted months earlier. The script parameters, e.g., specifications of flow rates, don't need to correspond to those previously specified, but they must correspond to the hardware and gas bottles planned months earlier and generally the chamber atmospheres as well by extension.

Non-script parameters specified in advance for s-Flame are listed here.

chamber fill pressure (psia)
chamber fill O2%
chamber fill He%
GC Sample (Before, After or No)
GC Sample Method (O2, Byproducts or Both)

Recirculation (After or No)
use HiBMS Camera?
HiBMS Binning
HiBMS Exposure (ms)
HiBMS Gain (V)
HiBMS Illumination (backlight)
use LLL-UV Camera?
LLL-UV Binning
LLL-UV Gain
LLL-UV Pulse Width
dataCamera Gain
dataCamera Focus
dataCamera Zoom
dataCamera Iris
dataCamera Filter
Intensified Camera
Intensified Camera Frame Rate
Intensified Camera Filter
Analog Camera Gain

D. Test Matrices: As-Run

Within constraints, the ACME tests conducted can use different parameter values than those specified in advance. While the planning is critically important, the plans themselves are not particularly useful for interpreting the data. This is because most tests were conducted with parameter values that differed in at least a few ways from those planned days earlier.

While it may seem obvious and simple enough to focus attention on the as-run parameter values (where an example s-Flame file name is as run20176A1.txt), that is unfortunately not the case for ACME because the as-run values were not always completely recorded, despite the implementation of automated recording. The most assured way to get the as-run parameter values is presumably from the test day's automated ACME log file, where an example name is 2020-06-24_15-10_ACME.log.

5. Nomenclature

ACME	Advanced Combustion via Microgravity Experiments (*project conducted in the CIR*)
ADP	Acceptance Data Package
AVP	avionics package
BRE	Burning Rate Emulator (*an ACME experiment*)
CASIS	Center for the Advancement of Science in Space
CCD	Charge-Coupled Device (*such as a camera sensor*)
CIA	Chamber Insert Assembly (*inserted into the CIR combustion chamber*)
CIR	Combustion Integrated Rack (*ISS facility in which ACME experiments are conducted*)
CFI	Cool Flames Investigation (*an MDCA experiment preceding ACME in the CIR*)
CFI-G	Cool Flames Investigation with Gases (*an ACME experiment selected by CASIS and NSF*)
CLD Flame	Coflow Laminar Diffusion Flame (*an ACME experiments*)

CM	Combustion Module (*space shuttle precursor to the CIR*)
CSV	comma-separated values
DataCam	Data Camera (*developed for the ACME experiments, characterized for pyrometry, etc.*)
E-FIELD Flames	Electric-Field Effects on Laminar Diffusion Flames (*an ACME experiment*)
EMI	electromagnetic interference
EMS	Experiment Mounting Structure
ERD	Engineering Requirements Document
Fiji	Fiji Is Just ImageJ (*a version of ImageJ preloaded with many plug-in features*)
Flame Design	*an ACME experiment*
FOMA	Fuel Oxidizer Management Assembly (*CIR's gas management subsystem*)
FOV	field of view
fps	frames per second
GC	Gas Chromatograph
GCIP	Gas Chromatograph Instrumentation Package (*a CIR subsystem*)
GIPOC	Glenn ISS Payload Operations Center (*also known as the TSC*)
GMT	Greenwich Mean Time
HiBMs	High Bit-depth Multi-spectral camera (*developed for the CIR facility*)
HOBJ	Halcon object (*binary uncompressed format; used for the DataCam*)
HV	high voltage
ImageJ	Image Processing and Analysis in Java (*open-source image processing software*)
IPSU	Image Processing and Storage Unit (*a CIR subsystem*)
IPSU-G	IPSU for Gigabit ethernet cameras
ISRD	Integrated Science Requirements Document (*for a set of experiments*)
ISS	International Space Station
js	JavaScript
LLL-UV	Low Light Level Ultraviolet camera ((*developed for the CIR facility*)
LSP	Laminar Soot Processes (*a shuttle experiment conducted in the Combustion Module*)
MDCA	Multi-user Droplet Combustion Apparatus (*CIR chamber insert preceding ACME*)
MFC	mass flow controller
Mist	Water-Mist Fire Suppression Experiment (*a Combustion Module experiment*)
MIUL	Materials Identification/Usage List
ms	millisecond
NSF	National Science Foundation
NTSC	National Television System Committee
OMA2	Optical Multi-channel Analyzer 2 (*open-source software for image processing and display*)
ops	operations
PaRIS	Passive Rack Isolation System
PIMS	Principal Investigator Microgravity Services
PMT	Photomultiplier Tube (*where ACME is equipped with three PMT modules*)
PSI	Physical Sciences Informatics System
PT	pressure transducer
psia	pounds per square inch
QD	quick disconnect
RDR	Requirements Definition Review
s-Flame	Structure and Response of Spherical Diffusion Flames (*an ACME experiment*)
SAMS	Space Acceleration Measurement System
SDP	Safety Data Package
slpm	standard liters per minute (*a measure of the mass or molar flow rate of a gas*)

SOFBALL	Structure Of Flame Balls At Low Lewis-number (*a Combustion Module experiment*)
SoFIE	Solid Fuel Ignition and Extinction (*project to be conducted in the CIR after ACME*)
SORD	Science Operations Requirements Document
SRD	Science Requirements Document
TFP	Thin-Filament Pyrometry (*a technique for making gas-phase temperature measurements*)
TIFF or TIF	Tagged Image File Format (*used for the CFI intensified, HiBMs, and LLL-UV cameras*)
TM	thermistor
TS	Transport Stream (*a video format*)
TSC	Telescience Support Center (*also known as the GIPOC*)
V	volt

